

Terraform Task - 2

Task Description:

Create 2 EC2 instances on 2 different regions and install nginx using terraform script.

The below Terraform file creates Amazon Linux EC2 NGINX server in Mumbai (ap-south-1) and Singapore (ap-southeast-1) and outputs both public URLs for both servers.

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
    }
  }
}

provider "aws" {
  alias   = "mumbai"
  region = "ap-south-1"
}

provider "aws" {
  alias   = "singapore"
  region = "ap-southeast-1"
}

data "aws_ami" "mumbai_server" {
  provider      = aws.mumbai
  most_recent   = true
  owners        = ["amazon"]

  filter {
    name     = "name"
    values   = ["al2023-ami-*-x86_64"]
  }
}

data "aws_ami" "singapore_server" {
  provider      = aws.singapore
  most_recent   = true
  owners        = ["amazon"]

  filter {
    name     = "name"
```

```
    values = ["al2023-ami-*-x86_64"]
  }
}

resource "aws_security_group" "mumbai_sg" {
  provider      = aws.mumbai
  name          = "mumbai-nginx-sg"
  description   = "Allow HTTP traffic to the Mumbai Nginx server"

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "singapore_sg" {
  provider      = aws.singapore
  name          = "singapore-sg"
  description   = "Allow HTTP traffic to the Singapore Nginx server"

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

```

resource "aws_instance" "mumbai_nginx" {
  provider           = aws.mumbai
  ami                = data.aws_ami.mumbai_server.id
  instance_type      = "t3.micro"
  associate_public_ip_address = true
  vpc_security_group_ids = [aws_security_group.mumbai_sg.id]

  user_data = <<-EOF
    #!/bin/bash
    dnf install -y nginx
    systemctl enable --now nginx
    echo '<h1>Nginx running in Mumbai</h1>' > /usr/share/nginx/html/index.html
  EOF

  user_data_replace_on_change = true

  tags = {
    Name = "Mumbai-Nginx-Server"
  }
}

resource "aws_instance" "singapore_nginx" {
  provider           = aws.singapore
  ami                = data.aws_ami.singapore_server.id
  instance_type      = "t3.micro"
  associate_public_ip_address = true
  vpc_security_group_ids = [aws_security_group.singapore_sg.id]

  user_data = <<-EOF
    #!/bin/bash
    dnf install -y nginx
    systemctl enable --now nginx
    echo '<h1>Nginx running in Singapore</h1>' > /usr/share/nginx/html/index.html
  EOF

  user_data_replace_on_change = true

  tags = {
    Name = "Singapore-Nginx-Server"
  }
}

output "public_urls" {
  value = {

```

```
mumbai    = "http://${aws_instance.mumbai.nginx.public_ip}"
singapore = "http://${aws_instance.singapore.nginx.public_ip}"
}
```

Steps to get the output:

1. Terraform init

```
● sathish@Jeeva-M1 two-region-nginx % terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.51.0...
- Installed hashicorp/aws v6.51.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

2. Terraform validate

```
● sathish@Jeeva-M1 two-region-nginx % terraform validate
Success! The configuration is valid.
```

3. Terraform plan

```
● sathish@Jeeva-M1 two-region-nginx % terraform plan
data.aws_ami.singapore_server: Reading...
data.aws_ami.mumbai_server: Reading...
data.aws_ami.mumbai_server: Read complete after 1s [id=ami-0eef9d717347382c1]
data.aws_ami.singapore_server: Read complete after 1s [id=ami-010ead48412d6f271]
```

```
Plan: 4 to add, 0 to change, 0 to destroy.
```

```
Changes to Outputs:
```

```
+ public_urls = {  
  + mumbai      = (known after apply)  
  + singapore   = (known after apply)  
}
```

4. Terraform apply

```
● sathish@Jeeva-M1 two-region-nginx % terraform apply --auto-approve  
data.aws_ami.singapore_server: Reading...  
data.aws_ami.mumbai_server: Reading...  
data.aws_ami.mumbai_server: Read complete after 0s [id=ami-0eef9d717347382c1]  
data.aws_ami.singapore_server: Read complete after 0s [id=ami-010ead48412d6f271]
```

```
Plan: 4 to add, 0 to change, 0 to destroy.
```

```
Changes to Outputs:
```

```
+ public_urls = {  
  + mumbai      = (known after apply)  
  + singapore   = (known after apply)  
}
```

```
aws_security_group.mumbai_sg: Creating...
```

```
aws_security_group.singapore_sg: Creating...
```

```
aws_security_group.mumbai_sg: Creation complete after 3s [id=sg-04330cbfdbba6b00b9]
```

```
aws_instance.mumbai_nginx: Creating...
```

```
aws_security_group.singapore_sg: Creation complete after 3s [id=sg-09064dabb470e6cd0]
```

```
aws_instance.singapore_nginx: Creating...
```

```
aws_instance.mumbai_nginx: Still creating... [00m10s elapsed]
```

```
aws_instance.singapore_nginx: Still creating... [00m10s elapsed]
```

```
aws_instance.mumbai_nginx: Creation complete after 12s [id=i-013b9882665ca1af9]
```

```
aws_instance.singapore_nginx: Creation complete after 13s [id=i-042830d1b33a77106]
```

```
Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
```

```
Outputs:
```

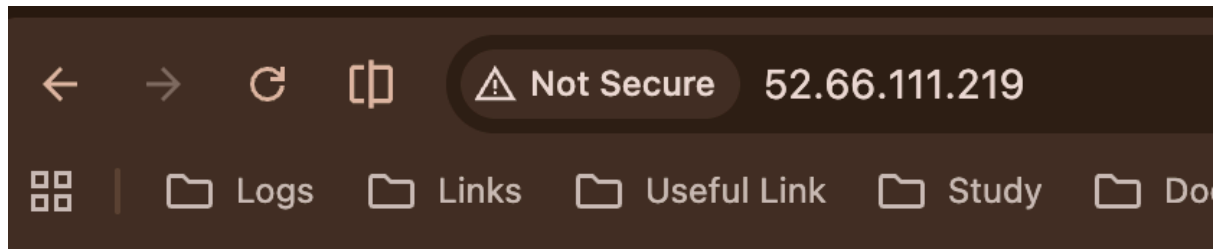
```
public_urls = {  
  "mumbai" = "http://52.66.111.219"  
  "singapore" = "http://13.212.224.97"  
}
```

```
❖ sathish@Jeeva-M1 two-region-nginx %
```

Screenshot of Mumbai instance in AWS console:

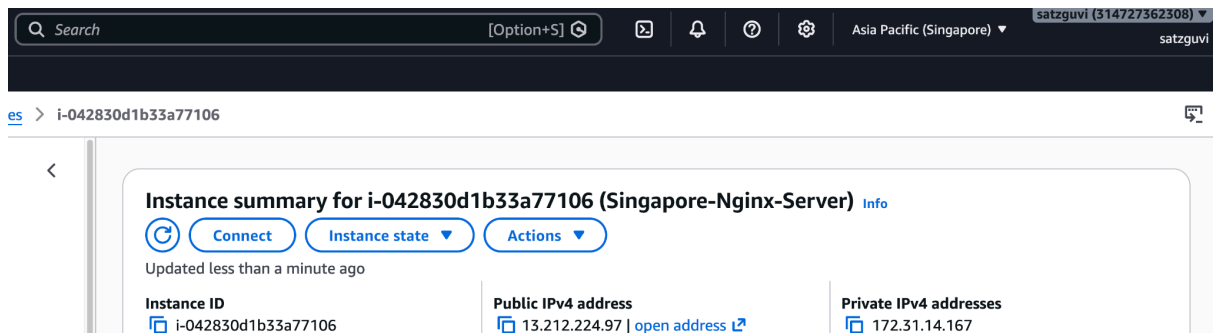
The screenshot shows the AWS Management Console interface. At the top, there's a search bar and navigation icons. Below that, the breadcrumb navigation shows 'ices > i-013b9882665ca1af9'. The main content area displays the 'Instance summary for i-013b9882665ca1af9 (Mumbai-Nginx-Server)'. It includes buttons for 'Connect', 'Instance state', and 'Actions'. Below these, it shows the instance ID 'i-013b9882665ca1af9', the public IPv4 address '52.66.111.219', and the private IPv4 addresses '172.31.33.109'. The instance is in the 'Running' state.

Screenshot of Mumbai NGINX server using public URL:

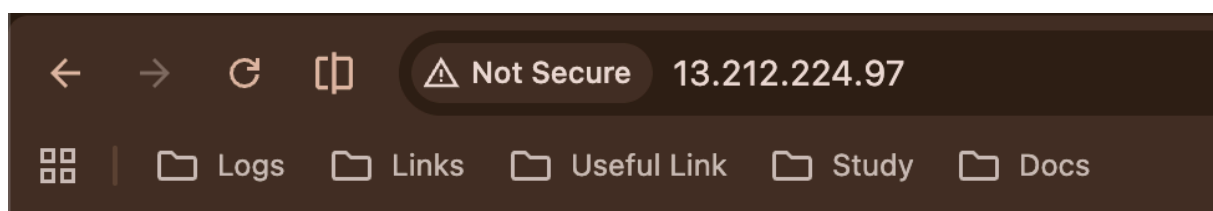


Nginx running in Mumbai

Screenshot of the Singapore instance in the AWS console:



Screenshot of Singapore NGINX server using public URL:



Nginx running in Singapore